Université Joseph Ki-Zerbo — IBAM
Master 1 Informatique
Module : Virtualisation et Cloud Computing

Projet RÉSULTATS-BF

Dossier d'architecture et de résultats

Une infrastructure résiliente et scalable,
entièrement décrite en code avec Vagrant

Groupe 1

KOURA L. Stéphane Ulrich
OUARMA Landry Ezeïkiel
SAWADOGO Ahmad Moussa
WAONGO Osiris Nouredine

Dépôt Git : https://github.com/Killmonger27/Resultats-BF-Groupe-1

Enseignant : Dr SOMDA Flavien Hervé

Année universitaire 2026–2027

26 octobre 2026

Table des matières

1 Contexte et objectifs **2**

2 Architecture et justification des choix **2**

3 Reproductibilité **4**

4 Exigences R1 à R8 **4**

5 Extensions **5**

6 Procédures d'exploitation **6**

7 Résultats des scénarios V1 à V6 **7**

8 Défauts révélés par les tests et corrections **10**

9 Limites connues et ce que nous ferions autrement **11**

10 Organisation de l'équipe et contributions **12**

A Annexes **13**

1 Contexte et objectifs

Nous avons construit RÉSULTATS-BF, une plateforme qui permet à un candidat de consulter son résultat d'examen par son matricule, et qui se reconstruit entièrement avec une seule commande Vagrant. Le sujet part d'un incident récurrent : le site tombe dans les dix minutes qui suivent la publication des résultats.

Deux exigences non fonctionnelles priment sur tout le reste :

— **Résilience** : la panne d'une machine ne doit pas interrompre le service ;
— **Scalabilité** : absorber plus de trafic se fait en ajoutant des machines, sans réécrire la configuration.

Le sujet évalue l'infrastructure autour d'une application volontairement minuscule, et notre capacité à prouver qu'elle tient. Chaque affirmation de ce rapport s'appuie donc sur une mesure archivée dans le dépôt Git du groupe, `https://github.com/Killmonger27/Resultats-BF-Groupe-1` (dossier `resultats/`).

Nous travaillons dans le sous-réseau du groupe 1, `192.168.56.0/24`, avec Vagrant, VirtualBox et la boîte `bento/ubuntu-24.04`. Le socle obligatoire compte cinq machines (base primaire, réplique, deux serveurs applicatifs, un répartiteur) dans 3 Go de mémoire. Nous avons réalisé deux extensions : **E1** (second répartiteur et adresse IP virtuelle) et **E4** (déploiement sans coupure).

2 Architecture et justification des choix

Chaque requête passe par une adresse IP virtuelle portée par l'un de deux répartiteurs, est distribuée à N serveurs applicatifs sans état, qui lisent leurs données sur un PostgreSQL primaire doublé d'une réplique. Tout est décrit dans le dépôt et construit par `vagrant up`.

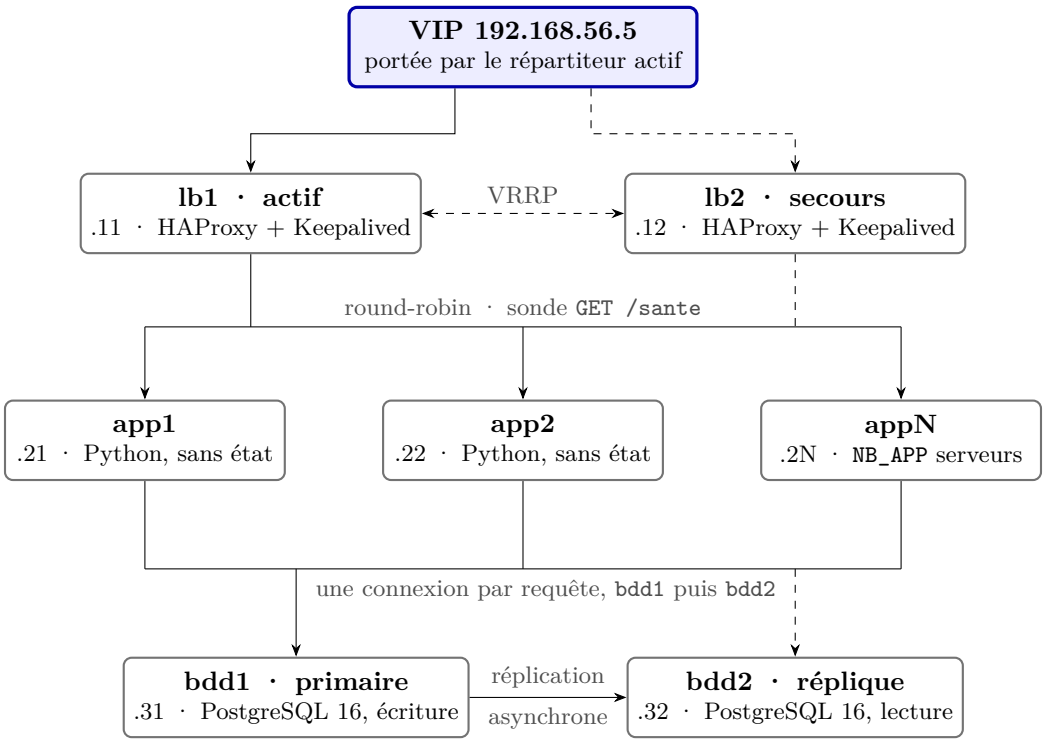


Figure 1 – Architecture de la plateforme. Trait plein : trafic normal ; tirets : secours (lb2) ou bascule (bdd2).

2.1 Inventaire des machines

Machine	Rôle	Adresse	RAM	Statut
VIP	adresse virtuelle, portée par le répartiteur actif	192.168.56.5	—	E1
lb1	répartiteur actif (HAProxy, Keepalived)	192.168.56.11	512 Mo	socle
lb2	répartiteur passif	192.168.56.12	512 Mo	E1
app1 à appN	serveurs applicatifs sans état ($N = 2$ par défaut)	192.168.56.21 à .2N	512 Mo	socle
bdd1	PostgreSQL primaire, lecture et écriture	192.168.56.31	768 Mo	socle
bdd2	réplique en flux, promouvable	192.168.56.32	768 Mo	socle

Le socle (lb1, deux serveurs applicatifs, bdd1, bdd2) occupe 3 072 Mo ; avec lb2, 3 584 Mo. Le réseau privé est `192.168.56.0/24` (groupe 1) ; aucun port des machines n'est redirigé vers l'hôte, hors le SSH de Vagrant.

2.2 Source unique de vérité

Le Vagrantfile décrit les machines dans une seule table et en écrit une copie dans `config/noeuds.yml` à chaque commande. Les scripts lisent ce fichier : la configuration du répartiteur, les adresses de la base vues par l'application et les noms utilisés par les scripts d'exploitation en dérivent. Aucune adresse de serveur applicatif n'est écrite en dur ; passer de 2 à 4 serveurs se fait avec `NB_APP=4 vagrant up`.

2.3 Répartiteur : HAProxy et Keepalived

— **HAProxy** répartit en tourniquet (`roundrobin`) sur le port 80 de la VIP et sonde chaque serveur par `GET /sante`. Un serveur est retiré après 2 échecs espacés d'1 s et réintégré après 2 succès ; la sonde dispose de 4 s pour répondre. Ainsi, seuls les serveurs dont la base répond reçoivent du trafic.

— **Keepalived** (VRRP en monodiffusion, annonce toutes les secondes) donne la priorité 101 à lb1 et 100 à lb2 : la VIP passe sur lb2 si lb1 tombe. HAProxy peut démarrer sur le répartiteur passif avant qu'il ne possède la VIP (`ip_nonlocal_bind`).

— Le socket d'administration de HAProxy sert aux déploiements sans coupure (extension E4).

2.4 Application

Le service est écrit en Python standard avec `psycopg2` (95 lignes, limite du sujet : 100). Il expose `/resultat`, `/sante` et `/info`, écoute sur `0.0.0.0:8080` et ne conserve aucun état : une connexion à la base par requête, aucune session, aucun fichier, aucun compteur. Toute sa configuration vient de l'environnement, écrit par `app.sh` dans un fichier de mode `0600`, et systemd le relance s'il s'arrête. Les adresses de `bdd1` et `bdd2` sont lues dans `noeuds.yml` ; la connexion exige un serveur qui accepte les écritures (`target_session_attrs=read-write`), donc `/sante` reste en 503 tant que la réplique n'est pas promue.

2.5 Données

PostgreSQL 16 (paquet Ubuntu 24.04) héberge la table `resultats` (matricule en clé primaire, nom, série, moyenne, admis), remplie de 50 000 lignes par `generate_series`. `bdd2` est une réplique en flux, asynchrone, avec un slot de réplication ; l'application n'a que le droit `SELECT`, un rôle distinct sert à la réplication, et `pg_hba.conf` n'ouvre que le sous-réseau privé en `scram-sha-256`. Les données restent sur le disque de la machine virtuelle, jamais dans le dossier synchronisé.

3 Reproductibilité

La plateforme entière se détruit et se reconstruit avec `vagrant destroy -f` puis `vagrant up`, sans aucune commande ensuite : trois reconstructions complètes ont duré 7 min 28 s, 9 min 8 s (quatre serveurs applicatifs) et 7 min 32 s, toutes avec le code de sortie 0.

— **Idempotence.** Chaque script réécrit ses fichiers en entier, jamais par ajout (`>>`). La configuration PostgreSQL n'est réécrite que si elle diffère et le service ne redémarre que dans ce cas ; le bloc ajouté à `pg_hba.conf` est balisé, retiré puis réécrit ; les rôles, la base et le slot de réplication ne sont créés que s'ils n'existent pas ; les 50 000 lignes ne sont insérées que si la table est vide ; la copie de `bdd2` n'est refaite que si elle n'est pas déjà une réplique.

— **Secrets.** Les mots de passe viennent de `.env`, ignoré par Git ; seul le modèle `.env.exemple` est versionné. L'application les reçoit dans un fichier de mode `0600`. La recherche des valeurs réelles dans l'historique et les fichiers suivis ne donne aucune occurrence.

— **Fins de ligne.** `.gitattributes` impose `eol=lf` aux fichiers texte et aux scripts, pour qu'un script écrit sous Windows ne casse pas dans la machine virtuelle.

— **Ordre de démarrage.** Le Vagrantfile déclare la base avant les serveurs applicatifs, puis les répartiteurs ; `replique.sh` attend jusqu'à deux minutes que `bdd1` réponde. L'application ne se connecte pas au lancement et supporte une base absente.

— **Connexion lente.** Chaque script installe ses paquets en une seule commande `apt-get`, pour ne rien télécharger deux fois.

Version de la boîte. Le dépôt épingle `bento/ubuntu-24.04` en version `202404.26.0`, qui n'existe en VirtualBox que pour les processeurs amd64. Nos mesures, faites sur un Mac Apple Silicon, ont utilisé la version `202510.26.0` (amd64 et arm64), sans modifier le dépôt.

4 Exigences R1 à R8

Les huit exigences obligatoires sont satisfaites et mesurées ; la bascule de la base (R6) n'a abouti qu'après la correction de deux défauts, décrits à la section 8. Les preuves sont dans `resultats/`.

Exigence	Résultat mesuré	Preuve
R1 reconstruction intégrale	3 reconstructions sans intervention : 7 min 28 s, 9 min 8 s ($N = 4$), 7 min 32 s ; test de fumée (`/sante`, 7 contrôles d'API, 50 000 lignes sur les deux bases) à chaque fois	`campagne2/contexte.txt`, `campagne2/r1_up_final_n2.log`
R2 idempotence	deux `vagrant provision` successifs sur app1, lb1, bdd1 et bdd2 : aucun écart de fichier, aucune erreur, PostgreSQL non redémarré	`campagne2/r2/`, `campagne1/r2/`

Exigence	Résultat mesuré	Preuve
R3 nombre de serveurs paramétrable	`NB_APP=4 vagrant up` depuis zéro : 4 lignes `server` engendrées dans HAProxy, 4 serveurs `UP`, répartition 5/5/5/5	`campagne2/contexte.txt`, `campagne1/observations.txt` [4] et [6]
R4 sonde de santé	`/sante` interrogée par HAProxy ; un serveur arrêté est écarté en 0,2 à 2,2 s (6 essais, seuil : moins de 5 s)	`campagne2/pannes/v2_*.log`, `campagne2/avant_correctifs/pannes_plateforme1/`
R5 réplication	une ligne écrite sur bdd1 est lue sur bdd2 ; une écriture sur bdd2 est refusée (`read-only transaction`)	`campagne1/observations.txt`
R6 bascule chronométrée	`scripts/bascule.sh` : bdd2 promue en 1,6 s, service de nouveau en 200 en 3,7 s, 100 lignes sur 100 retrouvées	`campagne2/bascule/`, `campagne2/observations.txt` [1] à [4]
R7 aucun secret dans Git	0 occurrence des deux mots de passe réels dans l'historique et les fichiers suivis ; `.env` ignoré, `.env.exemple` versionné	commande dans `docs/guide-verification.md`
R8 seul le répartiteur expose un port	aucune redirection de port dans le Vagrantfile ; seules celles du SSH de Vagrant existent, liées à `127.0.0.1`	`Vagrantfile`

5 Extensions

Deux extensions sont réalisées et démontrées, le minimum demandé : E1 (répartiteur redondant) et E4 (déploiement sans coupure). E2 (observabilité) et E3 (cache) ne sont pas réalisées.

5.1 E1 : répartiteur redondant

lb2 tourne en secours de lb1 avec Keepalived : quand lb1 s'arrête, la VIP passe sur lb2 et le service reste joignable à la même adresse. Sur 9 essais de `vagrant halt lb1` sous sonde de 0,1 s, 8 sont conformes : une seule requête perdue, entre 2,1 et 2,6 s après l'arrêt (seuil : moins de 5 s). Au retour de lb1, la VIP revient sur lui avec 0 ou 1 requête perdue. Le neuvième essai est non conforme (136 requêtes en erreur sur 81,5 s) ; il date d'une session où VirtualBox était instable, ne s'est pas reproduit et sa cause n'est pas établie.

5.2 E4 : déploiement sans coupure

`scripts/deploy.sh` met à jour les serveurs applicatifs un par un, depuis l'hôte :

1. il retire le serveur du répartiteur (`drain`, par le socket d'administration de HAProxy, sur chaque répartiteur en marche) ;
2. il attend que ses sessions actives tombent à zéro ;
3. `scripts/maj-app.sh` copie la nouvelle version et redémarre le service ;
4. il remet le serveur en service et attend que HAProxy le voie `UP` avant de passer au suivant.

Si une étape échoue, le déploiement s'arrête et un serveur dont la mise à jour a échoué reste retiré : la mauvaise version n'est jamais exposée. La démonstration `tests/e4_deploiement.sh 2.0` envoie la charge de 8 clients sur la VIP pendant que la version passe de 1.0 à 2.0 : sur 4 essais (5 247 à 5 521 requêtes), aucune requête n'a échoué, et chaque serveur a servi les deux versions successivement (`resultats/e4/`).

6 Procédures d'exploitation

Cinq procédures couvrent la vie de la plateforme ; chacune a été exécutée pendant la campagne de tests. Elles se lancent depuis la racine du dépôt.

6.1 Démarrer et reconstruire

```
cp .env.exemple .env              # renseigner DB_PASSWORD et REPL_PASSWORD
vagrant up                        # construit les 6 machines, la base d'abord
vagrant destroy -f && vagrant up  # reconstruction complète
```

Le `.env` ne contient que des lignes `CLE=valeur`, sans espace ni caractère spécial, car il est lu par le shell.

6.2 Ajouter un serveur applicatif

```
NB_APP=4 vagrant up app3 app4       # crée les deux serveurs
NB_APP=4 vagrant provision lb1 lb2  # régénère la configuration HAProxy
```

Sur une plateforme neuve, `NB_APP=4 vagrant up` suffit. La variable doit accompagner chaque commande Vagrant suivante tant que les 4 serveurs existent. Pour revenir à 2 : `NB_APP=4 vagrant destroy -f app3 app4`, puis `vagrant provision lb1 lb2`.

6.3 Basculer la base en cas de panne de bdd1

```
vagrant halt bdd1       # simulation de la panne
./scripts/bascule.sh    # promeut bdd2, attend le retour de /sante, affiche les durées
```

Les serveurs applicatifs n'ont rien à changer : ils essaient `bdd1` puis `bdd2` (adresses lues dans `noeuds.yml`) et trouvent la base promue. `bdd2` devient primaire et l'ancien primaire ne se réintègre pas : le retour à la normale est une reconstruction complète. Le script sort en erreur (code 1) si `/sante` ne revient pas.

6.4 Mettre à jour l'application sans coupure

```
./scripts/deploy.sh             # déploie le contenu de app/, serveur par serveur
./tests/e4_deploiement.sh 2.0   # démonstration sous charge, avec verdict OK ou KO
```

Prérequis : au moins 2 serveurs applicatifs `UP`, `socat` sur les répartiteurs, une base qui répond.

6.5 Pannes qui se résolvent seules

- **Un serveur applicatif s'arrête** : HAProxy le retire en quelques secondes ; `vagrant up appN` le réintègre environ 25 s après son démarrage.
- **lb1 s'arrête** : la VIP passe sur lb2, et revient sur lb1 au redémarrage.
- **bdd1 s'arrête sans bascule** : le service répond 503 puis revient à 200 environ 22 s après `vagrant up bdd1`.

7 Résultats des scénarios V1 à V6

Les six scénarios du sujet sont joués et mesurés ; deux donnent un résultat à nuancer (V3, V6) et un a révélé des défauts corrigés (V4). Les sorties brutes sont dans `resultats/`, et chaque scénario se rejoue avec le script de `tests/` cité.

Conditions. Mesures du 25 octobre 2026 sur un Apple M1 Pro (10 cœurs, 16 Go), VirtualBox 7.2.22, Vagrant 2.4.9, boîte `bento/ubuntu-24.04` 202510.26.0, deux serveurs applicatifs sauf mention. Le commit de base est `925678f`, auquel s'ajoutent les commits de la branche. `ab` et la sonde tournent sur l'hôte et partagent son processeur avec les machines virtuelles. Les pannes sont mesurées par une sonde qui interroge la VIP toutes les 0,1 s (abandon à 1 s) ; `t = 0` est le lancement de la commande de panne. Contexte complet : `resultats/campagne2/contexte.txt`.

7.1 V1 : répartition de charge

Objectif. Montrer que les N serveurs reçoivent le trafic à tour de rôle (R3).

```
for i in $(seq 20); do curl -s http://192.168.56.5/info; echo; done | sort | uniq -c
```

Rubrique	Contenu
Résultat brut	$N = 2$: 10 réponses `app1` et 10 `app2`. $N = 4$: 5 réponses par serveur, de `app1` à `app4`
Mesure relevée	écart $(\max - \min)/\text{total} = 0\,\%$ pour $N = 2$ et $N = 4$ (seuil : moins de 20 %)
Commentaire	le tourniquet est exact pour des requêtes séquentielles identiques. Source : `campagne1/observations.txt` [4]

7.2 V2 : panne d'un serveur applicatif

Objectif. Prouver que la sonde `/sante` écarte un serveur mort en moins de 5 s (R4).

```
SEULEMENT=v2 REP=3 ./tests/v2_v6_pannes.sh    # vagrant halt app1 à t+6 s, sous sonde de 0,1 s
```

Rubrique	Contenu
Résultat brut	essai 1 : 843 requêtes, 3 sans réponse, de t+2,6 s à t+4,8 s. Essai 2 : 851 requêtes, 3 erreurs (un 502, deux sans réponse), de t+2,3 s à t+3,6 s. Essai 3 : 860 requêtes, 2 erreurs, de t+2,3 s à t+2,5 s
Mesure relevée	période dégradée de 2,2 s, 1,3 s et 0,2 s ; 200 en continu dès t+5,8 s, t+4,6 s et t+3,5 s. Une première série de 3 essais a donné 2,2 s, 0,2 s et 1,3 s (seuil : moins de 5 s)
Commentaire	les erreurs sont isolées (aucune série continue) : la première survient quand le service s'arrête pendant l'extinction de la machine, HAProxy retire le serveur après 2 sondes ratées d'1 s. `app1` est réintégré tout seul environ 25 s après son redémarrage. Source : `campagne2/pannes/v2_*.log`

7.3 V3 : montée en charge

Objectif. Mesurer l'effet de serveurs supplémentaires sur le débit.

```
./tests/v3_charge.sh n2   # 5 passes d'ab -n 2000 -c 50 sur /resultat, après une passe de
    chauffe
```

Campagne (5 passes chacune)	Médiane (req/s)	Min	Max
$N = 2$, plateforme reconstruite	195,6	140,6	198,4
$N = 2$, après correction de la file d'écoute	194,9	133,4	196,0
$N = 2$, sous `caffeinate`, code final	180,4	143,6	181,1
$N = 4$, reconstruction à 4 serveurs	181,5	135,0	184,2

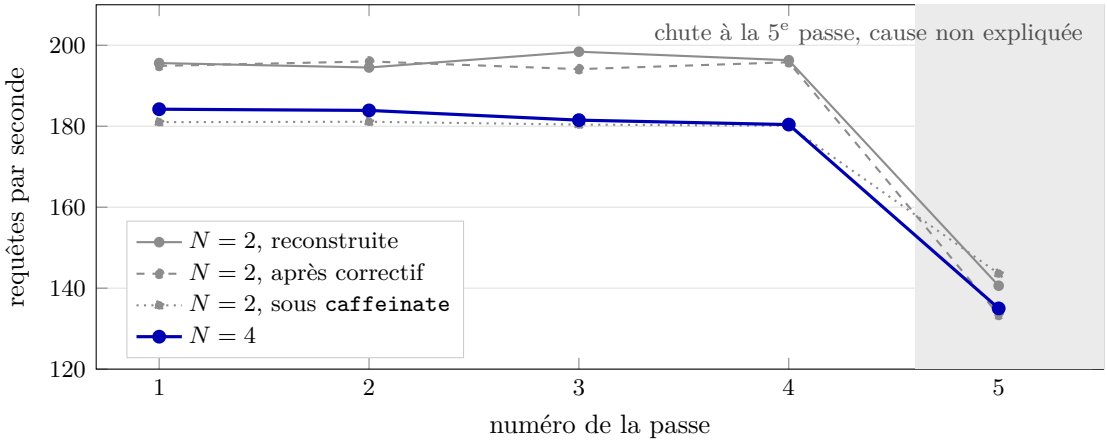


Figure 2 – Débit par passe : quatre serveurs ne font pas mieux que deux, et la cinquième passe chute dans chaque série (`ab -n 2000 -c 50`).

Rubrique	Contenu
Résultat brut	0 requête échouée dans toutes les passes ; p99 de 307 à 5 166 ms ; CPU de l'hôte de 20 à 41 %
Mesure relevée	le débit **n'augmente pas** avec les serveurs (médiane de 195,6 à 181,5 req/s en passant de 2 à 4). Il varie de 180 à 196 req/s d'une campagne à l'autre avec le même code
Commentaire	le goulot est mesuré : pendant 9 000 requêtes à $N = 4$, `bdd1` n'a plus que 10 % d'inactivité (36 % utilisateur, 39 % système, un seul processeur) alors que les serveurs applicatifs sont inactifs à 63 % et `lb1` à 87 %. La clé primaire sur `matricule` existe : ce n'est pas un défaut d'index. Le même matricule est interrogé à chaque requête, ce qui avantage la mesure. La 5e passe de chaque série chute (133 à 144 req/s, p99 de 3 à 5 s) : cause non expliquée. Sources : `campagne2/v3_*`, `campagne2/observations.txt` [5] à [9]

7.4 V4 : panne de la base primaire

Objectif. Vérifier que le service échoue proprement, puis chronométrer la bascule sur bdd2 (R6).

```
./tests/v4_panne_bdd.sh     # vagrant halt bdd1, observation du client, vagrant up bdd1
./tests/r6_bascule.sh       # vagrant halt bdd1 puis scripts/bascule.sh
```

Rubrique	Contenu
Résultat brut, sans promotion	pendant la panne, un 503 à chaque appel : d'abord le JSON de l'application (`{"erreur": "base indisponible"}`), puis la page de HAProxy (107 octets) quand `app1` et `app2` sont retirés. Client : 1 791 requêtes, 196 en 503, 6 sans réponse. `vagrant up bdd1` : 22,4 s. `bdd2` reste réplique avec 50 000 lignes ; 7 contrôles d'API OK ensuite

Rubrique	Contenu
Résultat brut, avec bascule	`bdd1` arrêtée en 4,6 s ; `bdd2` promue en 1,6 s ; `Service de nouveau en 200 : 3.7 s` ; 100 lignes sur 100 insérées juste avant la panne retrouvées sur `bdd2` ; écriture acceptée ; 7 contrôles OK ; les 4 serveurs `UP`
Mesure relevée	sans promotion : service de nouveau en 200 **22,5 s** après le début de `vagrant up bdd1`, sans intervention ; erreurs côté client de t+2,6 s à t+28,8 s. Avec bascule : **3,7 s** pour `bascule.sh`, 9,9 s depuis le début de l'arrêt de `bdd1`, 0 ligne perdue
Commentaire	jamais de page blanche ni d'erreur de connexion brute. Le premier essai de bascule a échoué : `bascule.sh` annonçait 63,8 s alors que le service n'est jamais revenu ; deux défauts en étaient la cause, corrigés (section 8). La réplication est asynchrone et l'arrêt de `bdd1` est gracieux : la perte d'une panne brutale n'est pas mesurée. Après la bascule, chaque requête met 0,6 à 2,0 s tant que `bdd1` est injoignable. Sources : `campagne2/v4/`, `campagne2/bascule/`, `campagne2/observations.txt` [1] à [4]

7.5 V5 : reconstruction

Objectif. Prouver que la plateforme se reconstruit sans geste manuel (R1).

```
./tests/r1_reconstruction.sh    # destroy -f puis up, chronométré, puis api.sh
```

Rubrique	Contenu
Résultat brut	3 reconstructions, code de sortie 0 à chaque fois, avec les lignes `bdd1 prête : 50000 lignes.`, `bdd2 en réplication : t`, `[lb1] MASTER prêt` et `[lb2] BACKUP prêt`. Test de fumée ensuite : `/sante` 200, 7 contrôles d'API OK, répartition 10/10
Mesure relevée	448 s (7 min 28 s) pour $N = 2$, 548 s (9 min 8 s) pour $N = 4$, 452 s (7 min 32 s) pour $N = 2$ avec le code final
Commentaire	la boîte était déjà téléchargée : ce sont des reconstructions « à chaud ». La destruction dure de 7 à 17 s, le `vagrant up` de 436 à 531 s ; la durée dépend de la connexion, qui télécharge les paquets à chaque fois. Sources : `campagne2/contexte.txt`, `campagne2/r1_up_final_n2.log`

7.6 V6 : panne du répartiteur (E1)

Objectif. Vérifier que le service reste joignable à la même adresse quand lb1 tombe.

```
SEULEMENT=v6 REP=2 ./tests/v2_v6_pannes.sh    # halt lb1 à t+6 s, sonde de 0,1 s, retour de lb1
```

Rubrique	Contenu
Résultat brut	essai 1 : 861 requêtes, une erreur à t+2,1 s, stable dès t+3,1 s ; au retour de lb1, 724 requêtes sans erreur. Essai 2 : 860 requêtes, une erreur à t+2,5 s, stable dès t+3,5 s ; au retour, 717 requêtes et une erreur. La VIP est sur lb2 pendant la panne, puis revient sur lb1
Mesure relevée	reprise en 2,1 s et 2,5 s, avec une seule requête perdue (seuil : moins de 5 s). Au total, 8 essais sur 9 conformes, entre 2,1 et 2,6 s ; 1 essai non conforme (136 erreurs sur 81,5 s), non reproduit

Rubrique	Contenu
Commentaire	l'annonce VRRP d'1 s fait basculer la VIP en environ 2 s. L'arrêt de la machine lb1 dure jusqu'à 64 s, mais la VIP a déjà basculé. L'essai non conforme date d'une session où VirtualBox était instable ; son journal brut est perdu et sa cause n'est pas établie. Sources : `campagne2/pannes/v6_*.log`, `campagne2/avant_correctifs/pannes_plateforme1/`, `campagne1/observations.txt` [10]

8 Défauts révélés par les tests et corrections

La campagne de mesures a révélé quatre défauts que la lecture du code n'avait pas montrés ; trois ont été corrigés dans le dépôt, le quatrième est une contrainte de l'environnement de mesure. Le deuxième vient d'un réglage que nous avions nous-mêmes introduit.

#	Défaut	Comment il a été trouvé	Correction
1	**File d'écoute de l'application à 5** (valeur par défaut du serveur HTTP Python) : sous charge, des connexions refusées étaient retransmises 1 à 3 s plus tard	V3 : le compteur du noyau `TcpExtListenOverflows` augmente de 57 sur app1 et de 61 sur app2 en 3 passes ; `Send-Q` vaut 5 sur le port 8080 ; 327 retransmissions de SYN sur lb1	`request_queue_size = 128` dans `app.py`, déployé sans coupure avec `deploy.sh`. Ensuite, aucun débordement sur 13 passes ; les p99 de 1,2 s disparaissent de la série
2	**Sonde HAProxy trop courte** : `inter 1s` sans `timeout check` donne 1 s de réponse à la sonde, alors que `/sante` met 1,6 à 2,05 s quand bdd1 est éteinte (délai de connexion avant de passer à bdd2). HAProxy rejetait des serveurs sains et la bascule n'aboutissait jamais	R6 : après la promotion de bdd2, `/sante` répond 200 en 1,6 s sur app1, mais HAProxy note `L7TOUT` après 1 003 ms ; 221 s d'erreurs côté client	`timeout check 4s` dans le modèle HAProxy. Les serveurs repassent `UP` aussitôt, le service est rétabli en 3,7 s
3	**Succès annoncé à tort par `bascule.sh`** : après 60 s sans réponse 200, il affichait quand même « Service de nouveau en 200 »	R6 : message de 63,8 s alors que `/sante` n'est jamais revenue à 200	le script distingue rétabli, non mesuré et échec (message d'erreur, code de sortie 1)
4	**Mise en veille du portable** (réglage à 1 minute) pendant des essais non surveillés : VirtualBox suspend les machines. Un essai de V4 a duré 852 s au lieu de 22 s	`systemd-analyze` donne 7 s de démarrage dans la VM, mais `VBox.log` montre 76 s de suspension et `pmset` cinq mises en veille	les essais finaux tournent sous `caffeinate` ; les essais invalidés sont refaits

Les corrections 2 et 3 touchent des fichiers qui ne sont pas ceux de l'application (modèle HAProxy, script de bascule) ; elles ont été isolées dans un commit séparé, pour être relues par leurs responsables. Sources : `campagne2/observations.txt` [1] à [10].

9 Limites connues et ce que nous ferions autrement

Le passage à l'échelle est la principale limite : ajouter des serveurs applicatifs n'augmente pas le débit, car la base unique, avec un seul processeur, est saturée. Les autres limites sont listées ci-dessous avec leur mesure.

9.1 Limites connues

— **Bascule ralentie.** Après la promotion de `bdd2`, chaque requête met 0,6 à 2,0 s tant que `bdd1` est injoignable, car l'application essaie `bdd1` en premier avec un délai de connexion de 2 s. Le service est correct mais lent jusqu'à la reconstruction.

— **Bascule manuelle et réplication asynchrone.** Il faut lancer `bascule.sh` ; l'ancien primaire ne se réintègre pas et un retour de `bdd1` après la bascule créerait deux primaires. Les 100 lignes insérées avant la panne ont toutes été retrouvées, mais l'arrêt était gracieux : la perte d'une panne brutale n'est pas mesurée.

— **Plus de lecture sur la réplique.** Avec `target_session_attrs=read-write`, le service est interrompu entre la panne de `bdd1` et la bascule, ce que le scénario V4 attend.

— **Une connexion par requête.** C'est le prix de l'absence d'état : `bdd1` consacre 39 % de son temps au système sous charge. Un pool de connexions réduirait ce coût, mais n'a pas été testé et doit être concilié avec la consigne « aucun état ».

— **Une passe de charge sur cinq chute** (133 à 144 req/s, p99 de 3 à 5 s), toujours la cinquième d'une série d'environ une minute. Cause non expliquée : nous avons écarté la file d'écoute (corrigée, aucun débordement), la mise en veille (le creux persiste sous `caffeinate`) et les connexions `TIME_WAIT` du client (aucune). Piste non vérifiée : app1 retransmet beaucoup de segments TCP (14 557 cumulés, 1 411 de plus sur 3 passes), ce qui évoque des pertes dans le réseau virtuel.

— **Un essai sur neuf de V6 est non conforme** (136 erreurs, 81,5 s), observé avant un redémarrage du Mac ; il ne s'est pas reproduit en 8 essais ensuite, et son journal brut est perdu.

— **Page 503 de HAProxy.** Quand tous les serveurs ont échoué la sonde, le client reçoit la page `503 Service Unavailable` de HAProxy, pas le JSON de l'application.

— **E4 suppose `socat` sur chaque répartiteur.** Sur une plateforme construite avant cette fonction, `deploy.sh` s'arrête sans rien modifier, avec un message clair ; `vagrant provision lb1 lb2` règle le cas.

— **Mesures à répéter.** Une campagne par scénario sur une seule machine ; le débit varie de 180 à 196 req/s d'une campagne à l'autre avec le même code.

— **README racine à corriger.** Son schéma d'origine parle de serveurs nginx et d'une réplication synchrone, alors que la plateforme utilise un service Python et une réplication asynchrone.

9.2 Ce que nous ferions autrement

— Mesurer d'abord la base : lui donner un second processeur ou tester un pool de connexions, avant d'ajouter des serveurs applicatifs.

— Basculer automatiquement la base, avec un outil comme Patroni si le budget mémoire de 3 Go le permet, et passer en réplication synchrone pour ne perdre aucune écriture.

— Lire sur la réplique et écrire sur le primaire, pour garder le service en lecture pendant une panne de `bdd1`.

— Lancer chaque campagne plusieurs fois sur la machine de référence, sans mise en veille, avec la même version de boîte pour tout le groupe.

— Mesurer une panne brutale de `bdd1` (arrêt immédiat de la machine) pour chiffrer la perte de données.

10 Organisation de l'équipe et contributions

Le travail s'est réparti selon les quatre rôles du sujet, à raison d'une branche Git par domaine fusionnée par pull request.

Membre	Rôle	Réalisé dans le dépôt
OUARMA Landry	Architecte réseau et répartition	Vagrantfile et table des machines, plan d'adressage, HAProxy (modèle et génération depuis `noeuds.yml`), Keepalived et VIP, extension E1
WAONGO Osiris	Ingénieur applicatif	service HTTP en Python, page de démonstration, provisionnement du service systemd
SAWADOGO Ahmad	Ingénieur données	PostgreSQL, jeu de 50 000 lignes, réplication en flux, procédure de bascule, modèle des secrets, documentation de la partie données
KOURA Ulrich	Ingénieur essais et observabilité	scénarios V1 à V6 et exigences R1 à R8 rejoués par scripts, campagne de mesures, extension E4, corrections du service, rapport

Notre branche. Elle ajoute deux commits (143 fichiers) : le premier corrige trois fichiers d'autres rôles après les tests de pannes (modèle HAProxy, installation de `socat`, script de bascule), le second contient les scripts de test, E4, les mesures, les corrections du service et la documentation. Un guide de vérification (`docs/guide-verification.md`) permet à la personne qui fusionne de rejouer les tests avant de fusionner.

Rotation des commits. Le sujet demande à chaque membre au moins un commit significatif hors de son domaine. L'historique permet de vérifier les travaux suivants, faits dans le domaine d'un collègue ou sur un livrable commun :

Auteur	Domaine touché	Modification
KOURA Ulrich	architecte (OUARMA Landry)	modèle HAProxy : sonde sur `/sante`, réglages `inter`, `fall`, `rise` et `timeout check 4s` ; `lb.sh` installe `socat`
KOURA Ulrich	données (SAWADOGO Ahmad)	`bascule.sh` distingue service rétabli, non mesuré et échec, et sort en erreur (code 1) si `/sante` ne revient pas
KOURA Ulrich	applicatif (WAONGO Osiris)	`app.py` et `app.sh` : 503 sans primaire en écriture, erreurs en JSON, file d'écoute à 128, version dans `/info` ; extension E4 (`deploy.sh`, `maj-app.sh`)
SAWADOGO Ahmad	architecte (OUARMA Landry)	Vagrantfile : exécute le script de chaque rôle (`scripts/<rôle>.sh`) dès qu'il existe
OUARMA Landry	données (SAWADOGO Ahmad)	`lib-pg.sh` : l'adresse de `bdd1` lue dans `noeuds.yml` ne dépend plus des fins de ligne du fichier (CRLF)
WAONGO Osiris	livrable commun (`README.md`, L1)	section « Application » : routes, configuration, installation et vérification du service (77 lignes)

A Annexes

A.1 Structure du dépôt

```
Vagrantfile    définition des VM et du réseau (source unique : config/noeuds.yml)
Makefile       commandes courantes (make help)
.env.exemple   modèle des secrets (cp .env.exemple .env)
app/           service HTTP (app.py, index.html, VERSION)
conf/          modèles HAProxy et Keepalived
config/        noeuds.yml, engendré par le Vagrantfile
scripts/       provisionnement des VM et procédures d'exploitation
tests/         scénarios de test et de mesure, outils dans tests/lib
resultats/     sorties brutes des mesures
docs/          dossiers par rôle, guide de vérification
```

A.2 Rejouer l'ensemble des tests

Les scripts se lancent depuis la racine du dépôt, un seul à la fois, dans cet ordre : la bascule et la reconstruction à 4 serveurs, qui modifient la plateforme, passent en dernier.

```
./tests/r1_reconstruction.sh                # R1 / V5 : reconstruction complète
./tests/api.sh                              # contrat de l'API : 7 contrôles
SEULEMENT=v2 REP=3 ./tests/v2_v6_pannes.sh  # V2 : panne d'un serveur
SEULEMENT=v6 REP=2 ./tests/v2_v6_pannes.sh  # V6 : panne du répartiteur
./tests/v4_panne_bdd.sh                     # V4 : panne de la base
./tests/e4_deploiement.sh 2.0               # E4 : mise à jour sous charge
./tests/r2_idempotence.sh                   # R2 : idempotence
./tests/v3_charge.sh n2                     # V3 : charge
NB_APP=4 ./tests/r1_reconstruction.sh       # R3 : reconstruction à 4 serveurs
./tests/r6_bascule.sh                       # R6 : bascule de la base (destructif)
./tests/r1_reconstruction.sh                # retour à 2 serveurs
```

Le guide `docs/guide-verification.md` donne pour chaque étape la durée, le résultat attendu et les seuils du sujet.

A.3 Sources des mesures

Dossier	Contenu
`resultats/campagne1/`	première campagne : charge `ab` ($N = 2$ et $N = 4$), pannes V2 et V6, idempotence R2, observations transcrites
`resultats/campagne2/`	campagne principale sur plateforme reconstruite : reconstructions, V2, V3, V4, V6, R2, R6 ; `avant_correctifs/` garde les essais antérieurs aux corrections ; `contexte.txt` donne le commit, la machine et les versions
`resultats/e4/`	sorties de la démonstration de mise à jour sous charge

Les journaux de sonde contiennent une ligne `horodatage code` par requête ; `tests/lib/analyse.py` les relit. Les relevés faits à l'écran, sans fichier brut, sont transcrits dans les fichiers `observations.txt` de chaque campagne.